

Computergestützte Physik 2 | SS 2025

Übung 7 | 20.05.2025

Abgabe: 27.05.2025, 16 Uhr

Vorbereitung zum 3. E-Test am 27.05.2025

Umfang des 3. E-Tests:

- Inhalte der Vorlesungen vom 12.05.2025 bis einschließlich 19.05.2025.
- Inhalte der Übungen 6-7, insbesondere Aufgabe 2.
- Code von **Aufgabe 1** aus dieser Übung (= Übung 7).
- Verständnisfragen des E-Tests beziehen sich auch auf **Aufgabe 2** aus dieser Übung.
- Regeln für den E-Test sind identisch zum vorherigen E-Test.

Hinweise zur Abgabe (bitte sorgfältig lesen!):

- Die Abgabe dieser Übung ist **freiwillig** und wird keine Punkte erzielen.
- Verwenden Sie das übliche Abgaben-Format.
- Sollten Sie sich nicht an dieses vorgegebene Schema halten, wird Ihre Abgabe nicht korrigiert.
- Die Skripte müssen mit aussagekräftigen print-Statements oder deutlich gekennzeichneten Plots eine eindeutige Zuordnung der Ergebnisse zu den jeweiligen Fragestellung ermöglichen.
- Es müssen alle verwendeten Datenfiles mit abgegeben werden, mit Ausnahme von durch uns zur Verfügung gestellten Daten. Die Skripte müssen sofort lauffähig sein. Es dürfen keine input Befehle verwendet werden.
- Andere Lösungen der Übungsaufgaben, z.B. mit iPython-Notebooks oder mit MAPLE, MATLAB, MATHEMATICA, ... können wir nicht akzeptieren.

Hinweis: Der Programmiereteil des 3. E-Tests wird sich auf diese Aufgabe (Aufgabe 1) beziehen.

Hinweis: Zur Selbstkontrolle. Die Ergebnisse x_k der ersten Iterations-Schritte k sind:

$$x_1 = [1.04138747, 1.08207086]$$

$$x_2 = [0.95876902, 0.91154631]$$

$$x_3 = [1.03965373, 1.0753438]$$

$$x_4 = [0.97646384, 0.94882795]$$

$$x_5 = [1.00373334, 1.00687187]$$

$$x_6 = [1.00001532, 1.00001537]$$

$$x_7 = [1.1.]$$

Nach $M = 5$ Schritten unterschreitet die Genauigkeit das gegebene ϵ mit einer normierten Differenz von $7.82e - 03$.

Aufgabe 1. (Von Hand gecodetes Newton-Verfahren)

Wir können das Newton-Verfahren nutzen, um eine Extremstelle der Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ zu berechnen. Hierfür ergibt sich die Iterationsvorschrift:

$$x_{k+1} = x_k - H_f(x_k)^{-1} \nabla f$$

Dabei ist H_f die Hessematrix.

In dieser Aufgabe betrachten wir die Funktion $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ mit

$$f(x, y) = (a - x)^2 + b(y - x^2)^2 + 0.1 \sin(3\pi x) \sin(3\pi y) \quad \text{mit} \quad a = 1, \quad b = 100.$$

Wir wollen zunächst $M = 2$ Schritte des Newton-Verfahrens mit Startpunkt

$$\mathbf{x}_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix} = \begin{pmatrix} 1.075 \\ 1.0625 \end{pmatrix}$$

durchführen.

- Definieren Sie eine Funktion, welche f , ∇f und die Hessematrix H_f mit `sympy` symbolisch berechnet und diese ausgibt.
- Für einen späteren Vergleich berechnen Sie die Lösung $\tilde{\mathbf{x}}$ des Optimierungs-Problems. Hier dürfen Sie `scipy.optimize` verwenden.

Hinweis: Achten Sie darauf, die Funktion f nicht erneut zu definieren, sondern z.B. mit `sp.lambdify` umzuwandeln.

- Schreiben Sie das Newton-Verfahren zur Extremstellenbestimmung einer Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ für $n = 2$ Dimensionen in `python` selber von Hand. Sie brauchen dabei ausschließlich die Pakete `numpy` und `sympy` verwenden.
 - Schreiben Sie Funktion `newton_schritt`, welche einen Iterationsschritt durchführt. Diese soll die **numerischen** Werte für $\mathbf{x}_k$, ∇f und $H_f(\mathbf{x}_k)$ als Parameter bekommen und den numerischen Wert $\mathbf{x}_{k+1}$ ausgeben. Diese Funktion soll ausschließlich `numpy` verwenden.
 - Schreiben Sie eine weitere Funktion `newton_verfahren`, welche den Startwert $\mathbf{x}_0$, die Funktion f , die 1. Ableitung ∇f , die Hessematrix H_f sowie eine Anzahl an Iterationsschritten M entgegen nimmt und das Ergebnis $\mathbf{x}_M$ ausgibt.
Beachten Sie: $\mathbf{x}_0$ und $\mathbf{x}_M$ sollen hier numerische Vektoren sein. f , ∇f , H_f hingegen sind symbolische Funktionen mit `sympy`.
 - Fügen Sie gerne sinnvolle `print`-Statements ein, um die Iteration zu beobachten.
- Führen Sie das Newton-Verfahren mit $M = 2$ Schritten durch.
- Nach wie vielen Schritten M erreicht der Newton-Algorithmus die Lösung mit einer Genauigkeit von $\epsilon = 10^{-2}$? Finden Sie dazu das kleinste k , mit dem

$$\|\mathbf{x}_k - \tilde{\mathbf{x}}\| \leq \epsilon$$

gilt. Hierfür können Sie gerne die Funktion `newton_verfahren` anpassen.

- Freiwillig, aber hilfreich:** Fügen Sie gerne eine weitere Funktion hinzu, welche die Funktion f grafisch für Sie sinnvoll und evtl. zusammen mit den $\mathbf{x}_k$ darstellt.

Hinweis: Eine Verständnisfrage des 3. E-Tests wird sich auf diese Aufgabe (Aufgabe 2) beziehen.

Aufgabe 2. (Optimierung eines Dark Matter Detektors)

Dem Beispiel der Vorlesung folgend sollen Sie einen Dark Matter Detektor nach MADMAX-Design optimieren. Verwenden Sie für diese Aufgabe den Code aus `boostfactor.py`.

Die Funktion `boostfactor(frequencies, distances)` bestimmt für einen Satz Scheibenabstände und Frequenzen den zugehörigen Boost-Faktor (Einheiten Hertz und Meter). Dieser Boost-Faktor soll im Folgenden optimiert werden.

Hinweis: Sie können die Funktion `np.random.seed` verwenden, um Zufallszahlen deterministisch generieren zu lassen und so vergleichbare Ergebnisse zu erhalten.

- a) Definieren Sie wie in der Vorlesung gezeigt eine Objective-Function, die das Minimum der Boost-Faktor Kurve bestimmt. Bedenken Sie dabei die Konvention, dass Optimierungsalgorithmen in der Regel Minimierungsfunktionen sind. Der Zustandsvektor x beschreibt die Abstände der Scheiben und soll der Funktion mit den zu optimierenden Frequenzen als Argumente übergeben werden.

- b) Implementieren Sie den Simulated Annealing Algorithmus, um optimale Plattenabstände des Detektors zu finden.

- Definieren Sie hierzu eine Funktion `find_neighbour(r, n)`, die einen Vektor $a\delta x$ bestimmt, wobei $a \in [0, r]$ gleichverteilt zufällig ist. Verwenden Sie die gegebene Funktion `space_uniform_rand` um δx als normierten, zufälligen, raumgleichverteilten Vektor zu bestimmen. n ist die Anzahl der Scheiben.

- Definieren Sie eine Funktion, die die "thermische" Wahrscheinlichkeit nach

$$\exp(-(f(y) - f(x))/t)$$

für zwei verschiedene Zustände x, y und die "Temperatur" t berechnet.

- Definieren Sie die Hauptfunktion. Sie sollte den Startvektor x_0 , die zu optimierenden Frequenzen, den Temperaturvektor T und die Schrittweite r als Argumente haben und den besten gefunden Lösungsvektor mit dem zugehörigen Objective Value zurückgeben. Die Anzahl der verwendeten Scheiben kann einfach aus der Länge des Startvektors festgelegt werden.

- c) Bestimmen Sie die Boost-Faktor Kurve eines Detektors mit 20 Scheiben in gleichen Abständen für den Frequenzbereich 21.9 bis 22.2 GHz und stellen Sie diese dar. Für alle Abstände wählen Sie hier 7.21 mm.

- d) Führen Sie die Optimierung des Detektors aus, mit dem Abständen aus c) als Startvektor. Als zu optimierende Frequenzen wählen Sie 10 Frequenzpunkte von 22.0 - 22.05 GHz. Probieren Sie verschiedene Werte für die Schrittweite und den Temperaturvektor aus. Schaffen Sie es eine Boost-Faktor Kurve mit einem besseren Objective Value als $-14\,000$ zu finden? Stellen Sie die optimierte Boost-Faktor Kurve zusätzlich zur Startkurve dar.

Hinweise: Als gute Schrittweite eignet sich 0.1 mm. Für zu große Schrittweiten (> 1 mm) können die Plattenabstände negativ werden, dies ist nicht physikalisch. Für den Temperaturvektor hat sich von 100 linear auf 0 fallend als sinnvoll erwiesen (siehe `np.linspace`).

- e) Zusätzliche Herausforderung - **NICHT PRÜFUNGSRELEVANT**: Implementieren Sie einen weiteren Optimierungs-Algorithmus Ihrer Wahl. Wie verhält er sich im Vergleich zu Simulated Annealing?